

Programovací jazyky a překladače

Přednáška 3, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

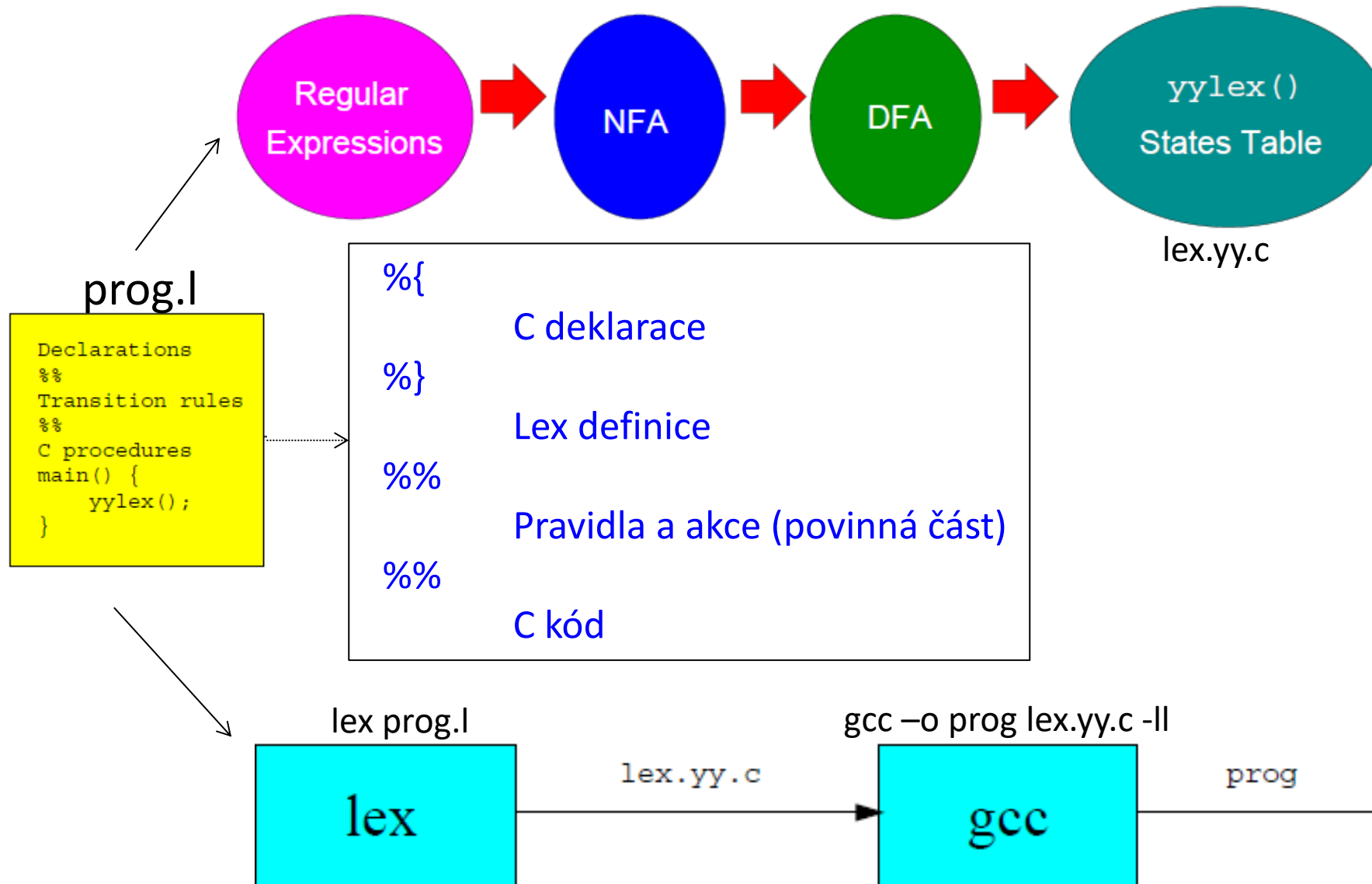
Témata

- Prezentace z oblasti formálních jazyků a gramatik
- Generátor lexikálních analyzátorů Lex
- Způsoby návrhu, překladu a vykonávání programovacích jazyků
 - HW a virtuální stroj
 - Zásobníkové a registrové procesory
 - Překladače a interprety
 - JIT, AOT: Java, Microsoft, Android
 - T diagramy

Prezentace

- Chomského hierarchie jazyků
- Gramatika jako generující formální struktura
- Automaty pro příjem jazyků - přehled a srovnání
- Lexikální analýza: principy a realizace konečným automatem
- Zásobníkové automaty
- Bezkontextové gramatiky

lex, generátor lexikálních analyzátorů



lex, tři jednoduché příklady

%%

.

\n

%%

```
int yywrap(void) {  
    return 1;  
}
```

```
int main(void) {  
    yylex();  
    return 0;  
}
```

Bez pravidel:
tisk symbolu na výstup

%%

Bez vzorů:
I/O kopie

DEFAULT

%%

.

\n

%%

```
int yywrap(void) {  
    return 1;  
}
```

```
int main(void) {  
    yylex();  
    return 0;  
}
```

Vykoná pravidlo
nebo {.. pravidla ..}

```
#define ECHO fwrite(yytext, yyleng, 1, yyout)
```

lex, příklady speciálních výrazů

Metacharacter	Matches
.	any character except newline
\n	newline
*	zero or more copies of the preceding expression
+	one or more copies of the preceding expression
?	zero or one copy of the preceding expression
^	beginning of line
\$	end of line
a b	a or b
(ab) +	one or more copies of ab (grouping)
[abc]	a or b or c (character class)
a{3}	3 instances of a
"a+b"	literal "a+b" (C escapes still work)

lex, složitější příklad

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}
```

← C/C++ deklarace

```
dgt [0-9]
```

← *lex*, definice

```
%%  
{dgt}+ return atoi(yytext);
```

← **Pravidla**

```
%%  
void main()  
{  
    int val, total = 0, n = 0;  
    while ( (val = yylex()) > 0 ) {  
        total += val;  
        n++;  
    }  
    if (n > 0) printf("ave = %d\n", total/n);  
}
```

← Hlavní program

lex, řešení konfliktů

Pokud aktuálnímu vstupu vyhovuje více předdefinovaných vzorů, postupuje *lex* následujícím způsobem:

- Vrátí **nejdelší** shodný token (např. $\leq$)
- V případě shodných vzorů téže délky vrátí ten, který je ve specifikaci uveden **dříve** (např. klíčová slova se definují před identifikátory)

Případy, kdy *lex* nenajde shodu, jsou obvykle ošetřeny **programátorsky**

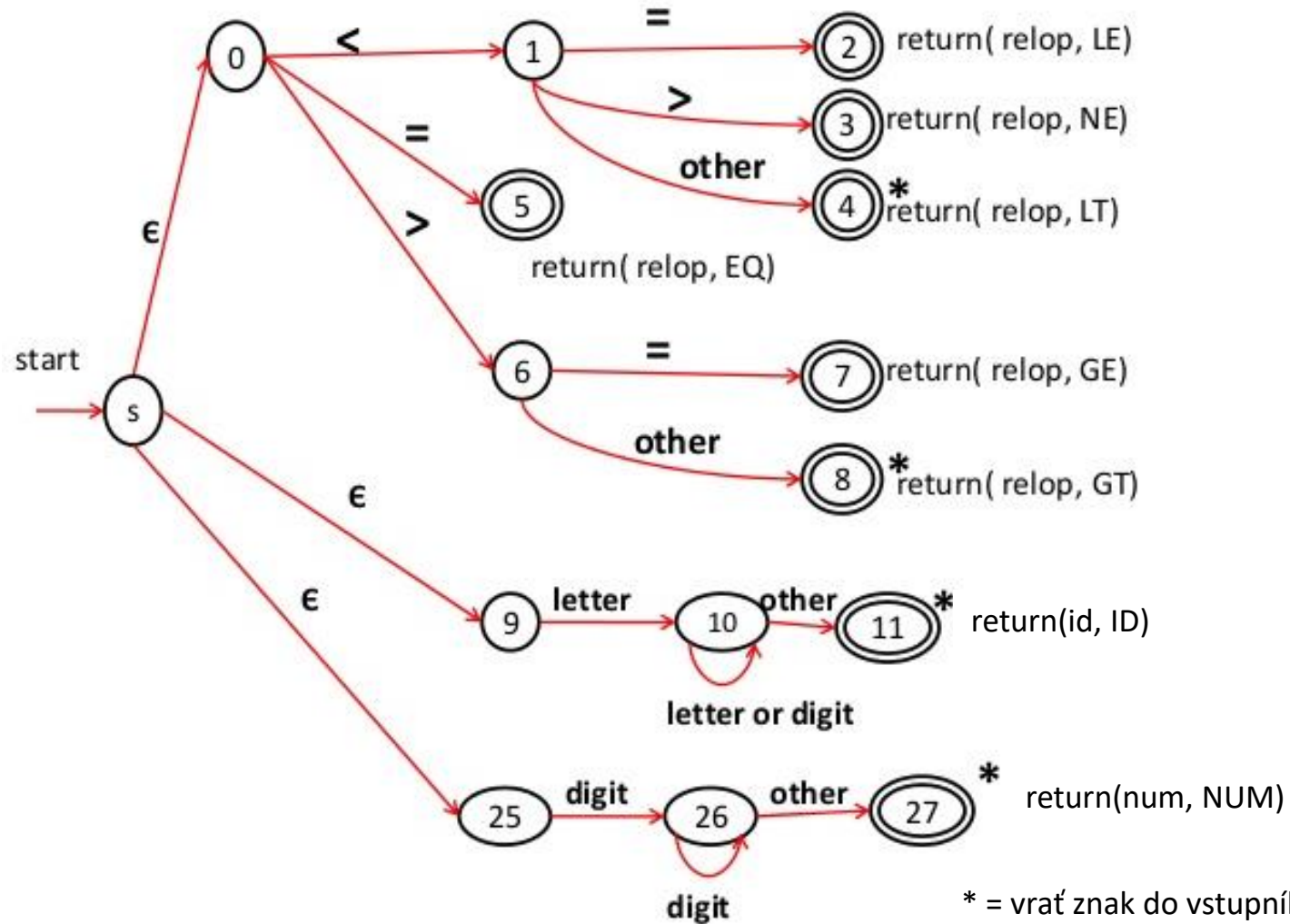
	Definice	Vstup	Token	Důvod
↓	if	if	if	→ je dříve
	ifthen	ifthen	ifthen	→ je delší

Problémy s potenciálně dlouhými tokeny, jak se scanneru mohou při nedostatečné definici jevit např. komentáře v C, mohou být řešeny **zařazením dodatečných stavů** (viz dále a ve cvičení).

lex, příklady speciálních funkcí

Name	Function
<code>char *yytext</code>	pointer to matched string
<code>int yyleng</code>	length of matched string
<code>int yylval</code>	value associated with token
<code>FILE *yyin</code>	input stream pointer
<code>FILE *yyout</code>	output stream pointer
<code>int yylex(void)</code>	call to invoke lexer, returns token
<code>char* yymore(void)</code>	return the next token
<code>int yyless(int n)</code>	retain the first n characters in yytext
<code>int yywrap(void)</code>	wrapup, return 1 if done, 0 if not done
ECHO	write matched string
REJECT	go to the next alternative rule
INITIAL	initial start condition
BEGIN	condition switch start condition

Konečný automat pro lexikální analýzu, příklad



Zdroj: AHO, A. V., LAM, M. S., SETHI, R., ULLMAN, J. D. Compilers: Principles, Techniques, and Tools.

Příklady komentářů v ANSI C

```
/* jednoduchý komentář */
```

```
/* také /* komentář */
```

```
/* komentář s * hvězdičkou uprostřed */
```

```
/* víceřádkový  
    komentář
```

```
*/
```

```
/* dva komentáře */ ..... /* na řádku */
```

```
* komentář s hvězdičkou na konci **/
```

Podobné problémy mohou vznikat i při práci s:

“textovými literály”.

RE pro komentáře v C

Takto?

`"/*" . "*" /*"`

`"/*"` textový literál

Nebo takto?

`"/*" ( . | \n ) "*" /*"`

`\n` nový řádek

Problém dlouhých vzorů:

`/* komentář * PROGRAM ( "text*/" ); ??`



Možné RE pro zpracování komentářů v C :

`"/*" ([^*] | \*+ [^*/] )* \*+ "/*"`

`[^*]` všechno mimo `*`; `\*` hvězdička

`"/*" ((( "/*" [^/] )? ) | [^*] ) "*" /*,`

`?` 0 nebo 1 výskyt

(f)lex jako stavový automat

- %s <STAV> : zahrnuté stavy (platí všechny včetně explicitně neoznačených)
 - %x <STAV> : vyloučené stavy (platí pouze stavy, označené návěštím)
-

```
%x comment
```

```
%%
```

```
int line_num = 1;
```

```
"/**"
```

```
BEGIN(comment);
```

```
<comment>[ ^* ]*
```

```
/* ignoruj cokoli mimo * */
```

```
<comment>"*" + [ ^* /\ ]*
```

```
/* ignoruj *, nenásledované / */
```

```
<comment>\n
```

```
++line_num;
```

```
<comment>"*" + "/"
```

```
BEGIN(INITIAL);
```

Příklad

%X S1, S2, S3

%%

"/"

<S1>"*"

<S2>[^*]

<S2>"*"

<S3>"*"

<S3>[^*/]

<S3>"/"

BEGIN(S1);

BEGIN(S2);

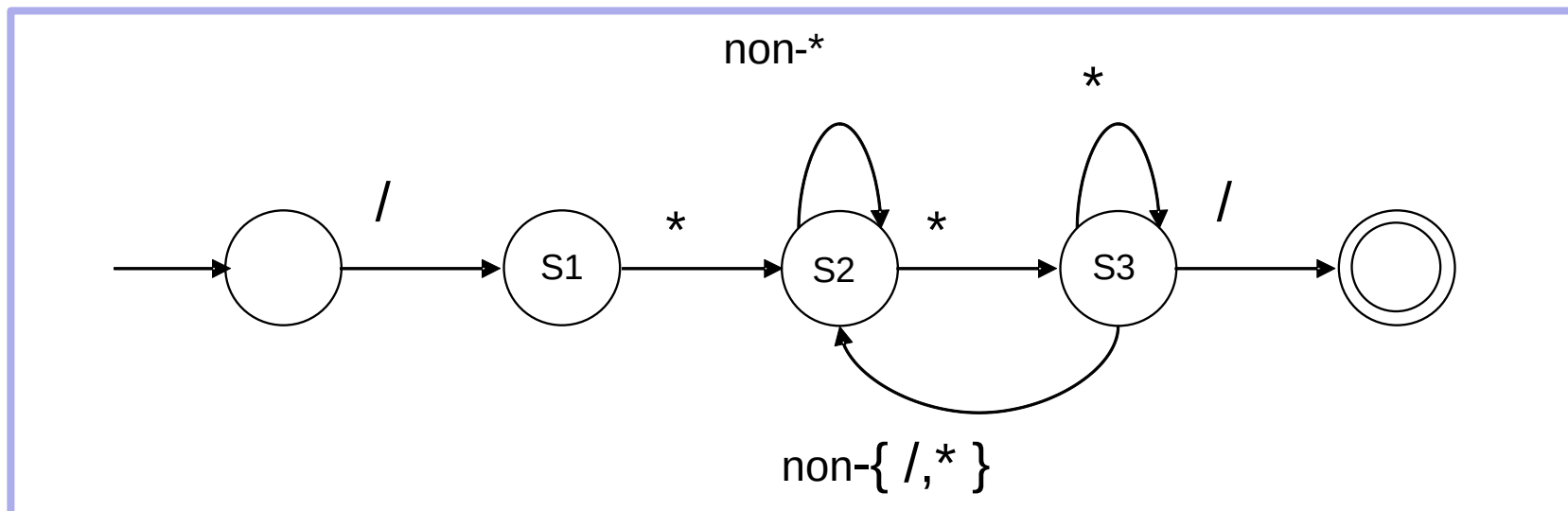
; /* zůstaň v S2 */

BEGIN(S3);

; /* zůstaň v S3 */

BEGIN(S2);

BEGIN(INITIAL);



Lexikální analýza

Miroslav Beneš
Dušan Kolář



Konstruktory překladačů

Miroslav Beneš
Dušan Kolář



Úkoly na příští týden

- Seznámit se s přednáškovými poznámkami a souvisejícími materiály
- Rezervovat individuální prezentace a hlídat si datum svého vystoupení
- V souladu s rezervační tabulkou průběžně připravovat a odevzdávat individuální prezentace